

SENAI - CRTI

Apostila

Manipulação de Dados com APIs (POST)

Projeto guiado: JSONPlaceholder

Interação com APIs — Sprint 3

Sumário

1. Introdução e pré-requisitos
2. Conceitos básicos
3. Passo a passo: criando o projeto pelo terminal
4. Código-fonte completo, arquivo por arquivo
5. Executando e testando a aplicação
6. Testes automatizados
7. Build de produção

1. Introdução e pré-requisitos

Nesta apostila você vai construir, do zero, uma aplicação Angular chamada JSONPlaceholder. O usuário preenche um formulário de cadastro de tarefa — título, descrição, prioridade, categoria, status e data limite — e, ao clicar em Salvar, a aplicação envia esses dados via POST para a API pública JSONPlaceholder, que responde confirmando a criação do recurso com um novo id.

Siga os capítulos na ordem. Cada comando de terminal e cada arquivo de código aparecem completos, prontos para copiar.

Pré-requisitos

- Node.js instalado (versão LTS mais recente).
- Angular CLI instalado globalmente.
- Um editor de código (ex.: VS Code).

Para instalar o Angular CLI globalmente, caso ainda não tenha:

```
$ npm install -g @angular/cli
```

Para conferir se tudo está pronto, rode os três comandos abaixo. Eles devem responder com um número de versão, sem erro:

```
$ node --version  
$ npm --version  
$ ng version
```

2. Conceitos básicos

Antes de programar, é importante entender o papel de cada peça que vamos usar.

HttpClient

É o serviço oficial do Angular para fazer requisições HTTP (GET, POST, PUT, DELETE...). Nesta apostila usamos o método `post()` para enviar os dados do formulário à API.

Método POST

O método HTTP POST é utilizado para criar um novo recurso no servidor. Diferente do GET, que apenas busca dados existentes, o POST envia informações no corpo (body) da requisição para que o servidor processe e crie um novo registro. A resposta padrão de sucesso é o código 201 Created, geralmente acompanhada do identificador (id) gerado para o novo recurso.

Payload

O payload é o corpo da requisição HTTP — o objeto de dados que o formulário envia ao servidor. No Angular, basta passar um objeto como segundo parâmetro de `http.post(url, payload)` que o `HttpClient` já cuida de serializá-lo em JSON e de definir o header `Content-Type: application/json` automaticamente.

Reactive Forms (FormGroup e FormControl)

Reactive Forms é a abordagem do Angular para formulários controlados via código TypeScript. Um `FormGroup` agrupa vários `FormControl`, um para cada campo do formulário, e cada `FormControl` pode ter validações (ex.: `Validators.required`). O método `form.getRawValue()` devolve um objeto com todos os valores atuais dos campos — exatamente o payload que enviamos para a API.

Service

Uma classe separada do componente, responsável apenas por se comunicar com a API. Isolar essa lógica em um service evita duplicar código e deixa a aplicação mais fácil de manter e testar.

Model (Interface)

Uma interface TypeScript que descreve o formato dos dados de uma tarefa (título, descrição, prioridade, categoria, status e data limite). Isso dá autocomplete e evita nomes de campo errados na hora de montar o payload.

Signals

Signals são variáveis reativas do Angular. Quando o valor de um signal muda (ex.: `signal.set(novoValor)` ou `signal.update(...)`), a tela é atualizada automaticamente, sem precisar de comandos manuais de renderização.

Observable e subscribe

O `HttpClient` não devolve o dado na hora — ele devolve um `Observable`, que representa uma resposta que vai chegar no futuro. Para receber esse valor quando ele chegar (ou tratar um erro), usamos `.subscribe({ next, error })`.

Zoneless (Angular 21+)

Desde a versão 21, o Angular não depende mais da biblioteca `Zone.js` para saber quando atualizar a tela. Em vez disso, a atualização acontece diretamente quando um `Signal` muda de valor. Isso deixa a aplicação mais leve e rápida, sem exigir nenhuma configuração extra — já vem assim por padrão em projetos novos.

3. Passo a passo: criando o projeto pelo terminal

Abra o terminal na pasta onde deseja criar o projeto e siga os comandos na ordem.

3.1 Criar o projeto Angular

```
$ ng new JSONPlaceholder --style=css --routing=false --ssr=false --defaults
```

O que cada opção faz:

- `--style=css`: usa CSS puro para os estilos.
- `--routing=false`: a aplicação tem uma única tela, não precisamos de rotas.
- `--ssr=false`: aplicação apenas no navegador (client-side), sem Server-Side Rendering.
- `--defaults`: aceita as demais respostas padrão automaticamente, sem perguntas interativas (o Angular 22 já cria o projeto zoneless por padrão).

Entre na pasta do projeto criado:

```
$ cd JSONPlaceholder
```

3.2 Gerar o model (interface) de Tarefa

```
$ cd
```

Esse comando cria o arquivo `src/app/models/tarefa.ts`, com uma interface `Tarefa` vazia. Vamos preencher os campos dela no próximo capítulo.

3.3 Gerar o service de Tarefa

```
$ ng generate service services/tarefa
```

Esse comando cria dois arquivos: `src/app/services/tarefa.ts` (o service) e `src/app/services/tarefa.spec.ts` (o arquivo de teste). Por padrão, o Angular CLI nomeia a classe gerada apenas como `Tarefa` — mas como já existe uma interface com esse mesmo nome, renomeie a classe do service para `TarefaService` (o nome do arquivo continua `tarefa.ts`). O service já nasce decorado com `@Service()` — o Angular 22 já o registra automaticamente na aplicação, sem precisar escrever `providedIn: 'root'`.

3.4 Estrutura final de pastas

Depois de todos os comandos, e com os arquivos preenchidos (próximo capítulo), a estrutura deve ficar assim:

```
src/  
  main.ts  
  app/  
    app.config.ts  
    app.ts  
    app.html  
    app.css  
    app.spec.ts  
    models/  
      tarefa.ts
```

```
services/  
  tarefa.ts  
  tarefa.spec.ts
```

4. Código-fonte completo, arquivo por arquivo

A seguir está o conteúdo final de cada arquivo. Abra cada um no editor e substitua o conteúdo gerado pelo Angular CLI pelo código abaixo.

4.1 src/main.ts

Ponto de entrada da aplicação. Inicializa (bootstrap) o componente principal App usando as configurações definidas em appConfig. Este arquivo é gerado pronto pelo Angular CLI — não precisa alterá-lo.

Arquivo: src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';  
import { appConfig } from './app/app.config';  
import { App } from './app/app';  
  
bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

4.2 src/app/app.config.ts

Configura os providers globais da aplicação. Aqui é onde habilitamos o HttpClient com provideHttpClient() — sem essa linha, o TarefaService não consegue fazer requisições HTTP.

Arquivo: src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';  
import { provideHttpClient } from '@angular/common/http';  
  
export const appConfig: ApplicationConfig = {  
  providers: [provideBrowserGlobalErrorListeners(), provideHttpClient()],  
};
```

4.3 src/app/models/tarefa.ts

Interface que descreve o formato dos dados de uma tarefa. O campo id é opcional (?) porque só existe depois que o servidor cria o recurso e o devolve na resposta.

Arquivo: src/app/models/tarefa.ts

```
export interface Tarefa {  
  id?: number;  
  titulo: string;  
  descricao: string;  
  prioridade: 'alta' | 'media' | 'baixa';  
  categoria: string;  
  status: 'pendente' | 'em-andamento' | 'concluida';  
  dataLimite: string;
```

```
}
```

4.4 src/app/services/tarefa.ts

O service concentra toda a comunicação com a API. O método criar recebe uma Tarefa (sem id ainda) e faz um POST para o endpoint da JSONPlaceholder, devolvendo um Observable tipado com <Tarefa> — a resposta chega com o mesmo objeto enviado, mais o id gerado pelo servidor.

Arquivo: src/app/services/tarefa.ts

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Tarefa } from '../models/tarefa';
@Injectable({
  providedIn: 'root'
})
export class TarefaService {
  private http = inject(HttpClient);
  private api = 'https://jsonplaceholder.typicode.com/posts';
  criar(tarefa: Tarefa) {
    return this.http.post<Tarefa>(this.api, tarefa);
  }
}
```

4.5 src/app/app.ts

O componente principal. Monta o FormGroup com um FormControl para cada campo da tarefa, guarda o estado da tela em três signals (tarefas, enviando, erro) e, no método salvar(), valida o formulário, monta o payload com form.getRawValue() e chama o service.

Arquivo: src/app/app.ts

```
import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from
 '@angular/forms';
import { TarefaService } from '../services/tarefa';
import { Tarefa } from '../models/tarefa';

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  private tarefaService = inject(TarefaService);

  form = new FormGroup({
    titulo: new FormControl('', { nullable: true, validators: [Validators.required]
  })),
    descricao: new FormControl('', { nullable: true, validators:
[Validators.required] }),
    prioridade: new FormControl<Tarefa['prioridade']>('media', { nullable: true }),
    categoria: new FormControl('', { nullable: true, validators:
[Validators.required] }),
    status: new FormControl<Tarefa['status']>('pendente', { nullable: true }),
```

```

    dataLimite: new FormControl('', { nullable: true, validators:
[Validators.required] }),
    });

    tarefas = signal<Tarefa[]>([]);
    enviando = signal(false);
    erro = signal('');

    // A JSONPlaceholder sempre responde ao POST com o mesmo id (101), então geramos
    // um id local exclusivo para cada tarefa criada nesta tela, evitando que duas
    // tarefas criadas colidam com o mesmo id (o que quebra o "track" da lista).
    private proximoIdLocal = Date.now();

    salvar() {
        if (this.form.invalid) {
            this.form.markAllAsTouched();
            this.erro.set('Preencha todos os campos obrigatórios.');
```

```

            return;
        }

        this.enviando.set(true);
        this.erro.set('');
```

```

        const tarefa: Tarefa = this.form.getRawValue();
```

```

        this.tarefaService.criar(tarefa).subscribe({
            next: (resposta) => {
                this.enviando.set(false);
                const tarefaCriada: Tarefa = { ...resposta, id: this.proximoIdLocal++ };
                this.tarefas.update((lista) => [tarefaCriada, ...lista]);
                this.form.reset({ prioridade: 'media', status: 'pendente' });
            },
            error: () => {
                this.enviando.set(false);
                this.erro.set('Erro ao criar a tarefa. Tente novamente.');
```

4.6 src/app/app.html

Template da tela: formulário reativo ligado ao form (título, descrição, prioridade, categoria, status e data limite), botão Salvar, mensagem de erro condicional (@if) e a lista de tarefas já criadas, renderizada com @for a partir do signal tarefas.

Arquivo: src/app/app.html

```

<div class="container">
  <h1>Cadastro de Tarefa</h1>

  <form class="form" [formGroup]="form" (ngSubmit)="salvar()">
    <label>
      Título
      <input type="text" formControlName="titulo" placeholder="Nome da tarefa" />
    </label>

    <label>
      Descrição
```

```

        <textarea formControlName="descricao" placeholder="Detalhes da
atividade"></textarea>
    </label>

    <div class="linha">
        <label>
            Prioridade
            <select formControlName="prioridade">
                <option value="baixa">Baixa</option>
                <option value="media">Média</option>
                <option value="alta">Alta</option>
            </select>
        </label>

        <label>
            Categoria
            <input type="text" formControlName="categoria" placeholder="Classificação da
tarefa" />
        </label>
    </div>

    <div class="linha">
        <label>
            Status
            <select formControlName="status">
                <option value="pendente">Pendente</option>
                <option value="em-andamento">Em andamento</option>
                <option value="concluida">Concluída</option>
            </select>
        </label>

        <label>
            Data limite
            <input type="date" formControlName="dataLimite" />
        </label>
    </div>

    <button type="submit" [disabled]="enviando()">
        {{ enviando() ? 'Salvando...' : 'Salvar' }}
    </button>
</form>

@if (erro()) {
    <p class="erro">{{ erro() }}</p>
}

@if (tarefas().length > 0) {
    <div class="lista">
        <h2>Tarefas criadas</h2>

        @for (tarefa of tarefas(); track tarefa.id) {
            <div class="card">
                <div class="card-topo">
                    <strong>{{ tarefa.titulo }}</strong>
                    <span class="badge">201 Created · id {{ tarefa.id }}</span>
                </div>
                <p>{{ tarefa.descricao }}</p>
                <p class="meta">
                    {{ tarefa.categoria }} · prioridade {{ tarefa.prioridade }} · status
                    {{ tarefa.status }} · até {{ tarefa.dataLimite }}
                </p>
            </div>
        }
    </div>
}

```

```
    }  
  </div>  
}  
</div>
```

4.7 src/app/app.css

Estilos da tela: layout centralizado, cartão branco do formulário, grade de dois campos por linha e os cartões da lista de tarefas criadas, com o selo verde indicando "201 Created" e o id devolvido pela API.

Arquivo: src/app/app.css

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
  font-family: Arial, sans-serif;  
}  
  
:host {  
  display: block;  
  background: #f5f7fb;  
  min-height: 100vh;  
}  
  
.container {  
  width: 100%;  
  max-width: 700px;  
  margin: 60px auto;  
  padding: 30px;  
}  
  
h1 {  
  text-align: center;  
  margin-bottom: 30px;  
  color: #222;  
}  
  
.form {  
  display: flex;  
  flex-direction: column;  
  gap: 16px;  
  background: white;  
  padding: 25px;  
  border-radius: 12px;  
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);  
  margin-bottom: 25px;  
}  
  
label {  
  display: flex;  
  flex-direction: column;  
  gap: 6px;  
  font-size: 14px;  
  color: #444;  
  font-weight: bold;  
}  
  
.linha {
```

```
    display: flex;
    gap: 16px;
}

.linha label {
    flex: 1;
}

input,
select,
textarea {
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
    font-size: 15px;
    font-family: inherit;
    font-weight: normal;
}

textarea {
    resize: vertical;
    min-height: 70px;
}

button {
    border: none;
    background: #2563eb;
    color: white;
    padding: 14px 24px;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
    font-size: 15px;
}

button:hover:not(:disabled) {
    background: #1d4ed8;
}

button:disabled {
    background: #93c5fd;
    cursor: not-allowed;
}

.erro {
    color: #dc2626;
    margin-bottom: 20px;
    font-weight: bold;
}

.lista h2 {
    margin-bottom: 15px;
    color: #111;
}

.card {
    background: white;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
    margin-bottom: 16px;
}
```

```
.card-topo {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 8px;
}

.badge {
  background: #dcfce7;
  color: #15803d;
  font-size: 12px;
  font-weight: bold;
  padding: 4px 10px;
  border-radius: 999px;
}

.card p {
  color: #444;
  margin: 4px 0;
}

.meta {
  font-size: 13px;
  color: #777;
}
```

5. Executando e testando a aplicação

Com todos os arquivos salvos, suba o servidor de desenvolvimento:

```
$ npm start
```

Ou, de forma equivalente:

```
$ ng serve
```

Abra o navegador em <http://localhost:4200> e teste os cenários abaixo.

Roteiro de teste

- Preencha todos os campos e clique em Salvar → deve aparecer um cartão em "Tarefas criadas" com o selo "201 Created · id ..." e os dados digitados.
- Deixe um campo obrigatório em branco (ex.: título) e clique em Salvar → deve aparecer a mensagem "Preencha todos os campos obrigatórios." sem nenhuma requisição sendo enviada.
- Cadastre várias tarefas seguidas → cada nova tarefa deve aparecer no topo da lista, com um id diferente devolvido pela API, e o formulário deve limpar automaticamente após cada envio.

6. Testes automatizados

O Angular 22 usa o Vitest como executor de testes por padrão. Os arquivos .spec.ts abaixo já verificam se o componente e o service são criados corretamente.

6.1 src/app/app.spec.ts

Arquivo: src/app/app.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { App } from './app';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
      providers: [provideHttpClient()],
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(App);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });

  it('should render title', async () => {
    const fixture = TestBed.createComponent(App);
    await fixture.whenStable();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector('h1')?.textContent).toContain('Cadastro de Tarefa');
  });
});
```

6.2 src/app/services/tarefa.spec.ts

Arquivo: src/app/services/tarefa.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { TarefaService } from './tarefa';

describe('TarefaService', () => {
  let service: TarefaService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient()],
    });
    service = TestBed.inject(TarefaService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
});
```

6.3 Rodando os testes

```
$ ng test --watch=false
```

O resultado esperado é a mensagem de todos os testes passando ("Tests ... passed").

7. Build de produção

Quando o projeto estiver pronto para publicar, gere a versão otimizada com:

```
$ ng build
```

Os arquivos finais são gerados na pasta dist/JSONPlaceholder, prontos para hospedar em qualquer servidor de arquivos estáticos.